

04-semantics

12. Channel Attribution Definitions

Channel attribution is the single most error-prone area of GA4 analysis, and Helios treats it as a first-class governed concern. The Decompose and Diagnose agents lean heavily on `channel_group` to explain funnel movement, so a wrong attribution rule silently corrupts every downstream finding. This section pins down the gotcha, the exact derivation, the attribution models, the channel grouping CASE logic, and the `dim_channels` dimension.

12.1 The traffic_source gotcha

In the GA4 BigQuery export, the event-level `traffic_source` STRUCT<name, medium, source> is **USER-LEVEL FIRST-TOUCH attribution**, not the source of the session that produced the event. It is stamped from the user's very first acquisition and copied onto every subsequent event for that `user_pseudo_id` for the entire export. A user acquired via Organic Search in November who returns in January via an Email campaign will still carry `traffic_source.medium = 'organic'` on the January events. Using event-level `traffic_source` for session-level channel analysis therefore systematically over-credits acquisition channels and under-credits re-engagement channels, and it can manufacture Simpson's-paradox confounds (mix-shift that looks like rate-change) — exactly what the core algorithm is designed to detect. **Rule: never use event-level `traffic_source` for session-scoped channel grouping.** It is permissible only as a documented fallback when session-scoped params are entirely NULL (rare), and as the basis for the genuinely user-level `dim_users.first_touch_channel`.

12.2 Session-scoped source / medium derivation

The session-correct source/medium lives in `event_params` on the session's own events (keys `source`, `medium`, `campaign`, `term`, `content`, `gclid`). These are populated chiefly on `session_start`, `first_visit`, and `page_view` events. The canonical rule is **first non-null value within the session ordered by `event_timestamp`**, where session = (`user_pseudo_id`, `ga_session_id`). Newer GA4 exports also expose `collected_traffic_source` STRUCT<...>; the obfuscated sample predates it, so Helios derives from `event_params` and documents `collected_traffic_source` as the preferred source if/when present.

```
-- int_ga4_sessionized: one row per (user_pseudo_id, ga_session_id) with session source/medium
WITH ev AS (
  SELECT
    user_pseudo_id,
    (SELECT ep.value.int_value FROM UNNEST(event_params) ep WHERE ep.key='ga_session_id') AS ga_session_id,
    event_timestamp,
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key='source') AS p_source,
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key='medium') AS p_medium,
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key='campaign') AS p_campaign,
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key='gclid') AS p_gclid
  FROM `bigquery-public-data.ga4_obfuscated_sample_ecommerce.events_*`
)
SELECT
  user_pseudo_id, ga_session_id,
  -- first non-null param ordered by event_timestamp (IGNORE NULLS picks earliest populated)
  COALESCE(ARRAY_AGG(p_source IGNORE NULLS ORDER BY event_timestamp LIMIT 1)[OFFSET(0)], '(direct)') AS session_source,
  COALESCE(ARRAY_AGG(p_medium IGNORE NULLS ORDER BY event_timestamp LIMIT 1)[OFFSET(0)], '(none)') AS session_medium,
  ARRAY_AGG(p_campaign IGNORE NULLS ORDER BY event_timestamp LIMIT 1)[OFFSET(0)] AS session_campaign,
  LOGICAL_OR(p_gclid IS NOT NULL) AS has_gclid
FROM ev
WHERE ga_session_id IS NOT NULL
GROUP BY user_pseudo_id, ga_session_id
```

Normalization rules applied before grouping: lowercase `source` / `medium`; empty string → NULL; missing source → '(direct)'; missing medium with direct source → '(none)'. These mirror GA4's own canonicalization so reconciliation against

`reconcile()` totals holds.

12.3 Attribution models

Helios computes three attribution models and exposes the model as an explicit choice so findings declare which they used:

- **First-touch** — credit the session/transaction to the channel of the user's first session ever (`ga_session_number = 1`). Stored on `dim_users.first_touch_channel` ; equals the (correctly-scoped) channel of the first session, not the event-level `traffic_source` .
- **Last-touch** — credit to the channel of the converting session itself. This is the Helios **default** for session-grained funnel and revenue diagnosis, because the funnel question is "what brought this session that converted."
- **Last-non-direct** — credit to the most recent non- (`direct`) channel within a lookback window (GA4 default 90 days), ignoring direct sessions. Used for revenue-at-risk attribution where direct traffic is treated as un-attributable re-entry.

Unless a finding states otherwise, `channel_group` on `fct_sessions` / `fct_funnel` / `fct_orders` is **last-touch session-scoped**.

12.4 GA4-style default channel grouping — classification rules

Derived strictly from session-scoped `session_source` / `session_medium` (+ `has_gclid` , `session_campaign`). Rules are evaluated top-to-bottom; first match wins. This mirrors GA4's default channel definitions.

channel_group	Rule (on lowercased session source/medium)
Direct	<code>medium IN ('(none)', '(not set)') AND source IN ('(direct)', '')</code>
Paid Search	<code>medium matches `^(cpc</code>
Paid Social	<code>medium matches `^(cpc</code>
Display	<code>medium matches `^(display</code>
Organic Search	<code>medium = 'organic'</code> OR source matches a search engine with non-paid medium
Organic Social	<code>medium IN ('social', 'social-network', 'social-media', 'sm', 'social network', 'social media')</code> OR source matches a social network with non-paid medium
Email	<code>medium IN ('email', 'e-mail', 'e_mail', 'newsletter')</code> OR source matches `email
Affiliates	<code>medium = 'affiliate'</code> OR <code>medium = 'affiliates'</code>
Referral	<code>medium IN ('referral', 'link')</code> (and not classified above)
Other	everything else / unclassifiable

12.5 channel_group CASE SQL

```
-- dim_channels resolver: applied in int_ga4_sessionized → propagated to facts
CASE
  -- Direct
  WHEN LOWER(session_medium) IN ('(none)', '(not set)', '') AND LOWER(session_source) IN ('(direct)', '')
    THEN 'Direct'
  -- Paid Search
  WHEN (REGEXP_CONTAINS(LOWER(session_medium), r'^(cpc|ppc|paid|paidsearch)$') OR has_gclid)
    AND REGEXP_CONTAINS(LOWER(session_source), r'google|bing|yahoo|duckduckgo|ecosia|baidu|yandex')
    THEN 'Paid Search'
  -- Paid Social
  WHEN REGEXP_CONTAINS(LOWER(session_medium), r'^(cpc|ppc|paid.*$')
    AND REGEXP_CONTAINS(LOWER(session_source),
      r'facebook|instagram|fb|twitter|x\.com|tiktok|linkedin|pinterest|reddit|snapchat|youtube')
    THEN 'Paid Social'
  -- Display
  WHEN REGEXP_CONTAINS(LOWER(session_medium), r'^(display|banner|expandable|interstitial|cpm)$')
```

```

    THEN 'Display'
-- Organic Search
WHEN LOWER(session_medium) = 'organic'
    OR REGEXP_CONTAINS(LOWER(session_source), r'google|bing|yahoo|duckduckgo|ecosia|baidu|yandex')
    THEN 'Organic Search'
-- Organic Social
WHEN LOWER(session_medium) IN ('social','social-network','social-media','sm','social network','social media')
    OR REGEXP_CONTAINS(LOWER(session_source), r'facebook|instagram|fb|twitter|tiktok|linkedin|pinterest|reddit|snapchat|youtube')
    THEN 'Organic Social'
-- Email
WHEN LOWER(session_medium) IN ('email','e-mail','e_mail','newsletter')
    OR REGEXP_CONTAINS(LOWER(session_source), r'email|newsletter')
    THEN 'Email'
-- Affiliates
WHEN LOWER(session_medium) IN ('affiliate','affiliates')
    THEN 'Affiliates'
-- Referral
WHEN LOWER(session_medium) IN ('referral','link')
    THEN 'Referral'
ELSE 'Other'
END AS channel_group

```

12.6 dim_channels

`dim_channels` is the conformed dimension table that materializes the grouping logic so it is authored once and reused everywhere. Its grain is **one row per channel_group** (~10 rows): the canonical channel groups enumerated once, each with its paid/organic flags and sort order. Facts carry their resolved `channel_group` and join to `dim_channels` on it, preventing ad-hoc CASE statements from drifting across models.

```

-- dims/dim_channels.sql (one row per channel_group; ~10 rows)
SELECT
    TO_HEX(MD5(channel_group)) AS channel_key,
    channel_group,
    CASE WHEN channel_group IN ('Paid Search','Paid Social','Display')
        THEN TRUE ELSE FALSE END AS is_paid,
    CASE WHEN channel_group IN ('Organic Search','Organic Social','Direct','Referral')
        THEN TRUE ELSE FALSE END AS is_organic,
    channel_group_order
FROM UNNEST([
    STRUCT('Direct' AS channel_group, 1 AS channel_group_order),
    STRUCT('Organic Search' AS channel_group, 2 AS channel_group_order),
    STRUCT('Paid Search' AS channel_group, 3 AS channel_group_order),
    STRUCT('Paid Social' AS channel_group, 4 AS channel_group_order),
    STRUCT('Organic Social' AS channel_group, 5 AS channel_group_order),
    STRUCT('Display' AS channel_group, 6 AS channel_group_order),
    STRUCT('Email' AS channel_group, 7 AS channel_group_order),
    STRUCT('Affiliates' AS channel_group, 8 AS channel_group_order),
    STRUCT('Referral' AS channel_group, 9 AS channel_group_order),
    STRUCT('Other' AS channel_group, 10 AS channel_group_order)
])

```

The per-session `channel_group` is resolved upstream in `int_ga4__sessionized` via the dbt macro `channel_group_case()` so the rules in 12.5 exist in exactly one place. `fct_sessions` carries `channel_group` and joins to `dim_channels` on `channel_group` (resolving `channel_key`, `is_paid`, `is_organic`, `channel_group_order`); the Diagnose agent reads `channel_group` only via `semantic-mcp`, never by re-deriving it.

13. Metric Definitions

This is the complete, authoritative metric catalog. Every metric the FOUNDATION names appears here with description, math, SQL expression, grain, numerator/denominator, applicable dimensions, filters, units, and gotchas. All metrics are session-grained at base (one row per `(user_pseudo_id, ga_session_id)` in `fct_sessions` / `fct_funnel`) unless noted; revenue metrics derive

from `fct_orders` (one row per `transaction_id`). Aggregation to any dimension is `SUM / COUNT` of the base, never an average of ratios — ratios are computed as `SUM(numerator)/SUM(denominator)` after grouping. This re-aggregation discipline is itself the defense against Simpson's paradox.

13.1 Catalog overview

metric	group	type	numerator	denominator	units	grain
sessions	volume	count	distinct (user_pseudo_id, ga_session_id)	—	count	session
users	volume	count	distinct user_pseudo_id	—	count	user
new_users	volume	count	users with first_visit/session_number=1	—	count	user
returning_users	volume	count	users – new_users	—	count	user
engaged_sessions	volume	count	sessions with session_engaged='1' OR engagement_time_msec >= 10000	—	count	session
engagement_rate	engagement	ratio	engaged_sessions	sessions	rate 0–1	session
view_item_sessions	volume	count	sessions with view_item	—	count	session
add_to_cart_sessions	volume	count	sessions with add_to_cart	—	count	session
begin_checkout_sessions	volume	count	sessions with begin_checkout	—	count	session
purchasing_sessions	volume	count	sessions with purchase	—	count	session
session_conversion_rate	funnel-rate	ratio	purchasing_sessions	sessions	rate 0–1	session
view_to_cart_rate	funnel-rate	ratio	add_to_cart_sessions	view_item_sessions	rate 0–1	session
cart_to_checkout_rate	funnel-rate	ratio	begin_checkout_sessions	add_to_cart_sessions	rate 0–1	session
checkout_to_purchase_rate	funnel-rate	ratio	purchasing_sessions	begin_checkout_sessions	rate 0–1	session
cart_abandonment_rate	funnel-rate	ratio	add_to_cart_sessions – purchasing_sessions	add_to_cart_sessions	rate 0–1	session
checkout_abandonment_rate	funnel-rate	ratio	begin_checkout_sessions – purchasing_sessions	begin_checkout_sessions	rate 0–1	session
transactions	revenue	count	distinct transaction_id	—	count	order
revenue	revenue	sum	sum purchase_revenue_in_usd	—	USD	order
gross_revenue	revenue	sum	sum purchase_revenue_in_usd	—	USD	order
net_revenue	revenue	sum	gross_revenue – sum refund_value_in_usd	—	USD	order
aov	revenue	ratio	revenue	transactions	USD	order
items_per_transaction	revenue	ratio	sum item quantity	transactions	items	order

metric	group	type	numerator	denominator	units	grain
revenue_per_session	efficiency	ratio	revenue	sessions	USD	session
revenue_per_user	efficiency	ratio	revenue	users	USD	user

13.2 Volume metrics

```
-- sessions
COUNT(DISTINCT session_key) AS sessions
-- users
COUNT(DISTINCT user_pseudo_id) AS users
-- new_users: first_visit event OR ga_session_number = 1
COUNT(DISTINCT IF(is_new_user, user_pseudo_id, NULL)) AS new_users
-- returning_users
COUNT(DISTINCT user_pseudo_id) - COUNT(DISTINCT IF(is_new_user, user_pseudo_id, NULL)) AS returning_users
-- step-reached session counts (booleans precomputed in fct_funnel)
COUNTIF(reached_view_item) AS view_item_sessions,
COUNTIF(reached_add_to_cart) AS add_to_cart_sessions,
COUNTIF(reached_begin_checkout) AS begin_checkout_sessions,
COUNTIF(reached_add_shipping_info) AS add_shipping_info_sessions,
COUNTIF(reached_add_payment_info) AS add_payment_info_sessions,
COUNTIF(reached_purchase) AS purchasing_sessions
-- session_key = TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))
```

Gotchas: `sessions` MUST be distinct on the composite `(user_pseudo_id, ga_session_id)` — `ga_session_id` is unique only within a user, never globally. `new_users` on the GA4 sample is best identified by the `first_visit` event because `user_id` is almost always NULL; `ga_session_number = 1` is the fallback. Step counts are session-presence booleans (did the session ever fire the event), not event counts.

13.3 Funnel-rate metrics

```
-- session_conversion_rate
SAFE_DIVIDE(COUNTIF(reached_purchase), COUNT(DISTINCT session_key)) AS session_conversion_rate,
-- step-to-step rates (denominator = prior step, NOT total sessions)
SAFE_DIVIDE(COUNTIF(reached_add_to_cart), COUNTIF(reached_view_item)) AS view_to_cart_rate,
SAFE_DIVIDE(COUNTIF(reached_begin_checkout), COUNTIF(reached_add_to_cart)) AS cart_to_checkout_rate,
SAFE_DIVIDE(COUNTIF(reached_add_shipping_info), COUNTIF(reached_begin_checkout)) AS checkout_to_shipping_rate,
SAFE_DIVIDE(COUNTIF(reached_add_payment_info), COUNTIF(reached_add_shipping_info)) AS shipping_to_payment_rate,
SAFE_DIVIDE(COUNTIF(reached_purchase), COUNTIF(reached_add_payment_info)) AS payment_to_purchase_rate,
SAFE_DIVIDE(COUNTIF(reached_purchase), COUNTIF(reached_begin_checkout)) AS checkout_to_purchase_rate,
-- abandonment
SAFE_DIVIDE(COUNTIF(reached_add_to_cart) - COUNTIF(reached_purchase), COUNTIF(reached_add_to_cart)) AS
  cart_abandonment_rate,
SAFE_DIVIDE(COUNTIF(reached_begin_checkout) - COUNTIF(reached_purchase), COUNTIF(reached_begin_checkout)) AS
  checkout_abandonment_rate
```

Gotchas: funnel step-to-step rates use the **immediately prior step** as denominator, not total sessions. The canonical macro funnel (`session_start` → `view_item` → `add_to_cart` → `begin_checkout` → `add_shipping_info` → `add_payment_info` → `purchase`) is monotonic only when steps are computed as "reached this step OR any later step"; Helios materializes `reached_*` booleans with this max-downstream rule so a session that purchases without a logged `begin_checkout` still counts as having reached checkout. Always wrap divisions in `SAFE_DIVIDE` to return NULL (not error) on zero denominators. When aggregating across dimensions, re-aggregate numerator and denominator separately — never average the per-segment rates.

13.4 Revenue metrics

```
-- transactions: distinct transaction_id from purchase events
COUNT(DISTINCT ecommerce.transaction_id) AS transactions,
-- revenue / gross_revenue
SUM(ecommerce.purchase_revenue_in_usd) AS revenue,
SUM(ecommerce.purchase_revenue_in_usd) AS gross_revenue,
```

```
-- net_revenue
SUM(ecommerce.purchase_revenue_in_usd) - SUM(COALESCE(ecommerce.refund_value_in_usd,0)) AS net_revenue,
-- aov
SAFE_DIVIDE(SUM(ecommerce.purchase_revenue_in_usd), COUNT(DISTINCT ecommerce.transaction_id)) AS aov,
-- items_per_transaction
SAFE_DIVIDE(SUM(ecommerce.total_item_quantity), COUNT(DISTINCT ecommerce.transaction_id)) AS items_per_transaction
```

Gotchas: revenue is taken once per `purchase` event from `ecommerce.purchase_revenue_in_usd`, NOT by summing `items[].item_revenue_in_usd` (which double-counts when both are present and excludes shipping/tax). `transaction_id` can repeat across shards if a purchase is re-logged — always `COUNT(DISTINCT)`. Use `*_in_usd` columns exclusively; the non-USD `purchase_revenue` / `price` are in the original currency and must never be mixed. `net_revenue` subtracts refunds, which in this dataset are sparse but must be coalesced to 0.

13.5 Efficiency metrics

```
-- revenue_per_session (RPS): revenue spread over ALL sessions, not just purchasing ones
SAFE_DIVIDE(SUM(revenue), COUNT(DISTINCT session_key)) AS revenue_per_session,
-- revenue_per_user (ARPU)
SAFE_DIVIDE(SUM(revenue), COUNT(DISTINCT user_pseudo_id)) AS revenue_per_user
```

Gotchas: RPS and ARPU denominators are **all** sessions/users in the window, not purchasers; this is intentional so the metric captures both conversion rate and AOV. Joining `fct_orders` to `fct_sessions` requires a left join from sessions so non-purchasing sessions remain in the denominator. RPS decomposes exactly as `session_conversion_rate × aov`, a relationship the Decompose agent exploits to attribute RPS movement to conversion vs basket-size.

13.6 Engagement metrics

```
-- engaged_sessions: GA4 engaged = session_engaged='1' OR engagement_time_msec ≥ 10000
COUNTIF(is_engaged_session) AS engaged_sessions,
-- engagement_rate
SAFE_DIVIDE(COUNTIF(is_engaged_session), COUNT(*)) AS engagement_rate
```

`is_engaged_session` is set in `int_ga4__sessionized` as `session_engaged = '1' OR engagement_time_msec ≥ 10000`, using the session-scoped `session_engaged` param. **Gotcha:** do not equate engagement with conversion; an engaged session need not purchase.

13.7 Cohort / retention metrics

These are computed by `stats-mcp` (`cohort_retention`, `rfm_segment`), but their grain and definition are governed here. Cohort = users grouped by week of `user_first_touch_timestamp`. `retention_rate(cohort, week_n)` = distinct users from the cohort with any session in week n divided by cohort size. RFM segments score Recency (days since last session), Frequency (distinct sessions), Monetary (sum `revenue`) into quintiles per `user_pseudo_id`. **Gotcha:** retention denominators are the original cohort size (fixed), never the surviving population, so retention is monotonically non-increasing.

14. Semantic Layer Design

The semantic layer is the heart of Helios's anti-hallucination guarantee. The PRINCIPLE is **grounding over generation**: the LLM never authors raw SQL or computes a statistic — it composes governed metric and dimension definitions through `semantic-mcp`, which is the ONLY path to SQL. Every column name, every join, every formula in this section comes from a registry of YAML definitions that an engineer owns, versions, and tests. If the LLM references a metric or dimension that is not in the registry, `build_query` raises before any SQL is generated — making "0 hallucinated columns/metrics (100% governed SQL)" a structural property, not a hope.

14.1 YAML schema for metrics and dimensions

Each metric and dimension is a YAML document with a fixed, validated schema. The fields:

```
# Field contract for a metric definition
name:      # snake_case canonical id (must match FOUNDATION exactly)
label:     # human display label
description: # one-line semantics, used in the exec brief glossary
type:      # one of: count | sum | ratio | derived
entity:    # grain entity: session | user | order | order_item
grain:     # physical base model this resolves against (e.g. fct_funnel)
agg:       # aggregation for additive metrics: count_distinct | sum | countif
sql:       # for count/sum: the additive SQL expression
numerator: # for ratio: name of numerator metric (must exist in registry)
denominator: # for ratio: name of denominator metric (must exist in registry)
expr:      # for derived: expression over other registered metrics
filters:   # list of governed filter predicates always applied
format:    # rendering: integer | percent_1dp | usd | decimal_2
dimensions: # whitelist of dimension names this metric may be sliced by
owner:     # accountable engineer / team
version:   # semver of this definition
```

```
# Field contract for a dimension definition
name:      # snake_case canonical id (must match FOUNDATION dimension list)
label:     # display label
description: # semantics
type:      # categorical | temporal | boolean
entity:    # session | user | order | item
sql:       # column or expression resolving the dimension on its entity
format:    # display format
owner:
version:
```

14.2 Example metric YAML entries

```
metrics:
- name: sessions
  label: Sessions
  description: Distinct (user_pseudo_id, ga_session_id) pairs in the window.
  type: count
  entity: session
  grain: fct_funnel
  agg: count_distinct
  sql: "session_key" # TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))
  filters: []
  format: integer
  dimensions: [device_category, operating_system, browser, country, region,
               channel_group, source, medium, campaign, landing_page,
               is_new_user, day, week, session_number_bucket]
  owner: analytics-eng
  version: 1.2.0

- name: add_shipping_info_sessions
  label: Add Shipping Info Sessions
  description: Sessions that reached the add_shipping_info step (max-downstream).
  type: count
  entity: session
  grain: fct_funnel
  agg: countif
  sql: "reached_add_shipping_info"
  filters: []
  format: integer
  dimensions: [device_category, operating_system, browser, country, region,
               channel_group, source, medium, campaign, landing_page,
               is_new_user, day, week, session_number_bucket]
  owner: analytics-eng
  version: 1.0.0
```

```

- name: add_payment_info_sessions
  label: Add Payment Info Sessions
  description: Sessions that reached the add_payment_info step (max-downstream).
  type: count
  entity: session
  grain: fct_funnel
  agg: countif
  sql: "reached_add_payment_info"
  filters: []
  format: integer
  dimensions: [device_category, operating_system, browser, country, region,
               channel_group, source, medium, campaign, landing_page,
               is_new_user, day, week, session_number_bucket]
  owner: analytics-eng
  version: 1.0.0

- name: purchasing_sessions
  label: Purchasing Sessions
  description: Sessions that reached the purchase step.
  type: count
  entity: session
  grain: fct_funnel
  agg: countif
  sql: "reached_purchase"
  filters: []
  format: integer
  dimensions: [device_category, operating_system, browser, country, region,
               channel_group, source, medium, campaign, landing_page,
               is_new_user, day, week, session_number_bucket]
  owner: analytics-eng
  version: 1.1.0

- name: session_conversion_rate
  label: Session Conversion Rate
  description: Share of sessions that purchased (purchasing_sessions / sessions).
  type: ratio
  entity: session
  grain: fct_funnel
  numerator: purchasing_sessions
  denominator: sessions
  filters: []
  format: percent_1dp
  dimensions: [device_category, operating_system, browser, country, region,
               channel_group, source, medium, campaign, landing_page,
               is_new_user, day, week, session_number_bucket]
  owner: analytics-eng
  version: 2.0.0

- name: revenue_per_session
  label: Revenue per Session (RPS)
  description: Revenue spread across all sessions; equals session_conversion_rate * aov.
  type: derived
  entity: session
  grain: fct_funnel
  expr: "SAFE_DIVIDE({revenue}, {sessions})"
  filters: []
  format: usd
  dimensions: [device_category, channel_group, country, day, week, is_new_user]
  owner: analytics-eng
  version: 1.0.0

```

14.3 Example dimension YAML entries

```

dimensions:
- name: channel_group
  label: Channel Group
  description: GA4-style default channel grouping from session-scoped source/medium.
  type: categorical

```

```

entity: session
sql: "channel_group" # resolved via dim_channels join, never re-derived ad hoc
format: string
owner: analytics-eng
version: 1.3.0

- name: device_category
label: Device Category
description: device.category of the session (desktop / mobile / tablet).
type: categorical
entity: session
sql: "device_category"
format: string
owner: analytics-eng
version: 1.0.0

```

14.4 The registry and how build_query composes validated SQL

The metric and dimension YAML files compile at load time into an in-memory **registry**: two dictionaries keyed by `name`, with referential integrity checks (every `numerator` / `denominator` / `expr` token must resolve to a registered metric; every entry in a metric's `dimensions` whitelist must be a registered dimension on a compatible entity). `semantic-mcp.build_query(metric, dims, filters, window)` is a deterministic resolver — not an LLM — that composes SQL strictly from these definitions:

```

def build_query(metric: str, dims: list[str], filters: dict, window: str) → str:
    m = REGISTRY.metrics[metric] # KeyError → hard fail, no hallucination
    for d in dims:
        if d not in m["dimensions"]: # dimension not whitelisted for metric
            raise SemanticError(f"{d} not permitted for {metric}")
    dim_sql = [REGISTRY.dimensions[d]["sql"] + f" AS {d}" for d in dims]
    where = compile_window(window) + compile_filters(filters, REGISTRY) # only governed predicates
    if m["type"] in ("count", "sum"):
        select = f"AGG[{m['agg']}]({m['sql']}) AS {metric}"
    elif m["type"] == "ratio":
        num, den = REGISTRY.metrics[m["numerator"]], REGISTRY.metrics[m["denominator"]]
        select = (f"SAFE_DIVIDE(AGG[num['agg']]({num['sql']}), "
            f"AGG[den['agg']]({den['sql']})) AS {metric}")
    elif m["type"] == "derived":
        select = expand_expr(m["expr"], REGISTRY) + f" AS {metric}"
    grp = f"GROUP BY {' '.join(str(i+1) for i in range(len(dims)))}" if dims else ""
    return f"SELECT {' '.join(dim_sql + [select])} FROM {GRAIN[m['grain']] WHERE {where} {grp}"

```

Why this prevents hallucinated columns: the LLM only ever passes string names. It can never emit a column, table, or formula directly. If it invents `conversion_pct`, the registry lookup fails loudly and the Critic agent flags it; the model retries with a real metric. Physical column names live exclusively in `sql` fields owned by analytics engineers, so a GA4 schema change is fixed in one YAML file, not across prompts. Every generated query is then `dry_run`-checked for cost/schema and `reconcile`-checked against canonical totals before results are trusted — verify-then-trust.

14.5 Governance, ownership, versioning

Each definition carries `owner` and semver `version`. Definitions live in `models/semantic/*.yaml` under code review; CI (GitHub Actions: `dbt build` + tests + eval harness) compiles the registry, runs referential-integrity checks, and fails the build on any dangling reference or schema drift. A breaking change to a formula bumps the major version and is recorded in the Memory store so prior diagnoses remain interpretable against the definition that produced them. The Critic agent additionally checks that a finding's cited metric `version` matches the run's registry version.

14.6 Mapping to dbt semantic layer / MetricFlow

The registry maps 1:1 onto dbt's semantic layer. Each `entity` / `grain` becomes a MetricFlow **semantic model** over the corresponding fact (`fct_funnel`, `fct_orders`); additive metrics map to MetricFlow `measures` (`agg: sum|count_distinct`); `type: ratio` maps to a `ratio` metric (numerator/denominator); `type: derived` maps to a `derived` metric over other

metrics; dimension `sql` maps to semantic-model `dimensions`. Helios ships a thin custom resolver (14.4) so it runs identically with or without a dbt Cloud Semantic Layer endpoint — the YAML is the single source of truth either way.

14.7 Worked example

`build_query('session_conversion_rate', ['device_category'], window='last_28d')` resolves: metric `session_conversion_rate` (ratio, grain `fct_funnel`, numerator `purchasing_sessions` [countif `reached_purchase`], denominator `sessions` [count_distinct of the session key]); dimension `device_category` (whitelisted); window `last_28d` → the trailing 28 `event_date` shards relative to the run date. Generated SQL:

```
SELECT
  device_category AS device_category,
  SAFE_DIVIDE(
    COUNTIF(reached_purchase),
    COUNT(DISTINCT session_key)
  ) AS session_conversion_rate
FROM `helios.marts.fct_funnel`
WHERE event_date BETWEEN
  FORMAT_DATE('%Y%m%d', DATE_SUB(CURRENT_DATE(), INTERVAL 28 DAY))
  AND FORMAT_DATE('%Y%m%d', CURRENT_DATE())
GROUP BY 1
```

This SQL contains zero column names the LLM chose; every token traces to a versioned YAML definition, dry-run validated and reconciled before any finding built on it ships.